

WEEK-7 SKILL

Task1 : Process Synchronization and Child Process Monitoring Using waitpid()

Aim : To create multiple child processes and use the waitpid() system call to synchronize and monitor child processes, obtain their termination status, and ensure that the parent process correctly waits for its children.

Step 1 — Create the C program

waitpid_demo.c

```
GNU nano 7.2 waitpid_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2;
    int status;

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        printf("Child 1 started. PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 terminating.\n");
        exit(10);
    }

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork failed");
        return 1;
    }
}
```

```

    if (child2 == 0)
    {
        printf("Child 2 started. PID = %d\n", getpid());
        sleep(4);
        printf("Child 2 terminating.\n");
        exit(20);
    }

    printf("Parent process. PID = %d\n", getpid());

    waitpid(child1, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child 1 finished with exit status: %d\n",
               WEXITSTATUS(status));
    }

    waitpid(child2, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child 2 finished with exit status: %d\n",
               WEXITSTATUS(status));
    }

    printf("Parent: All child processes completed.\n");

    return 0;
}

```

Step 2 — Compile and run the program

```

sainath@LAPTOP-GJ94VP0B:~/OSSP$ pwd
/home/sainath/OSSP
sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano waitpid_demo.c
sainath@LAPTOP-GJ94VP0B:~/OSSP$ gcc waitpid_demo.c -o waitpid_demo
sainath@LAPTOP-GJ94VP0B:~/OSSP$ ./waitpid_demo
Parent process. PID = 663
Child 1 started. PID = 664
Child 2 started. PID = 665
Child 1 terminating.
Child 1 finished with exit status: 10
Child 2 terminating.
Child 2 finished with exit status: 20
Parent: All child processes completed.
sainath@LAPTOP-GJ94VP0B:~/OSSP$

```

Observation:

Multiple child processes were successfully created using `fork()`. The `waitpid()` system call was used by the parent process to wait for specific child processes. The termination status of each child was obtained using `WIFEXITED()` and `WEXITSTATUS()`. The parent process waited for all child processes to complete before terminating

Result:

Process synchronization and child process monitoring were successfully implemented using `waitpid()`. The parent process correctly synchronized with the child processes and obtained their termination statuses.

Task2 : To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Aim : To understand how Linux uses the PATH environment variable to search for executable commands, verify executable permissions, handle missing commands, and test command resolution.

Step 1 — Create the C program

path_search.c

```
GNU nano 7.2 path_search.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main()
{
    char *path;
    char command[100];

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("Enter command to search: ");
    scanf("%99s", command);

    char *path_copy = malloc(strlen(path) + 1);

    if (path_copy == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }
}
```

```

strcpy(path_copy, path);

char *dir = strtok(path_copy, ":");
while (dir != NULL)
{
    char full_path[500];

    snprintf(full_path, sizeof(full_path), "%s/%s", dir, command);

    if (access(full_path, X_OK) == 0)
    {
        printf("Command found: %s\n", full_path);
        printf("Executable permission: Yes\n");

        free(path_copy);
        return 0;
    }

    dir = strtok(NULL, ":");
}

printf("Command not found or not executable.\n");

free(path_copy);

return 0;
}

```

Step 2 — Compile the program

```

sainath@LAPTOP-GJ94VP0B:~/OSSP$ nano path_search.c
sainath@LAPTOP-GJ94VP0B:~/OSSP$ gcc path_search.c -o path_search
sainath@LAPTOP-GJ94VP0B:~/OSSP$ ./path_search
Enter command to search: ls
Command found: /usr/bin/ls
Executable permission: Yes
sainath@LAPTOP-GJ94VP0B:~/OSSP$ |

```

Step 3 — Test a missing command

```

sainath@LAPTOP-GJ94VP0B:~/OSSP$ ./path_search
Enter command to search: abcdef
Command not found or not executable.
sainath@LAPTOP-GJ94VP0B:~/OSSP$ |

```

Step 4 — Check the PATH variable directly

```
sainath@LAPTOP-GJ94VP0B:~/OSSP$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr
/local/games:/usr/lib/wsl/lib:/mnt/c/Python314/Scripts:/mnt/c/Python314:/m
nt/c/Program Files/Eclipse Adoptium/jdk-21.0.9.10-hotspot/bin:/mnt/c/Program
Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common F
iles/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Ja
va/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/W
bem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/
OpenSSH:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/Java/jdk-21/bin:/
mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/nodejs:/mnt/c/Program
Files/Git/cmd:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/
sainath
.local/bin: LAPTOP-GJ94VP0B
LAPTOP-GJ94VP0B VS Code/bin:/mnt/c/
LAPTOP-GJ94VP0B : LAPTOP-GJ94VP0B 2026.1.2/bi
n: LAPTOP-GJ94VP0B :/
snap/bin
sainath@LAPTOP-GJ94VP0B:~/OSSP$ |
```

Step 5 — Verify executable permissions

```
sainath@LAPTOP-GJ94VP0B:~/OSSP$ ls -l /usr/bin/ls
-rwxr-xr-x 1 root root 142312 Aug 25 15:09 /usr/bin/ls
sainath@LAPTOP-GJ94VP0B:~/OSSP$ |
```

Step 6 — Final command-resolution test

```
sainath@LAPTOP-GJ94VP0B:~/OSSP$ ./path_search
Enter command to search: gcc
Command found: /usr/bin/gcc
Executable permission: Yes
sainath@LAPTOP-GJ94VP0B:~/OSSP$ |
```

Observation:

The PATH environment variable was successfully retrieved using `getenv()`. The program parsed the PATH into individual directories and searched each directory for the specified command. Executable permissions were verified using `access()`. Existing commands such as `ls` and `gcc` were successfully located, while a missing command was correctly reported as not found or not executable.

Result:

The PATH variable retrieval, directory parsing, executable searching, permission verification, missing-command handling, and command resolution were successfully implemented and tested in Ubuntu.

